



The fastest, ultra-realistic voice AI platform

CARTESIA[✦]

01

Intro



Zubin Pratap

♦ Head of DevRel, Cartesia

Previously

DevRel Eng Manager @ Chainlink Labs

Software Engineer @ Google

Ex-Founder

Corporate Lawyer for 15 years

<https://linkedin.com/in/zubinpratap>

<https://x.com/@zubinpratap>



Cartesia is an AI Research & Product Lab

Model / infrastructure layer

Model / infrastructure layer		
Sonic Text-to-speech	Ink Speech-to-text	René Speech-to-speech full duplex model coming 2026
Fastest streaming TTS 40ms time to first audio	Fastest streaming STT 90ms time to complete transcript	→ Context-aware responses with long-living memory
Adherence to complex transcripts Acronyms, initialisms, IDs	Optimized for real-world settings Background noises, disfluencies, accents	→ Context-aware responses with long-living memory
Preferred by human raters Naturalness, disfluencies, accent similarity		→ Natural backchanneling and interruptions
		→ Best for use cases needing high emotional awareness

Application layer

Line

Voice agent platform

Built for today's AI teams:

Code-first environment to build, deploy, and test your voice agents quickly with one-shot code generations and reasoning templates

Fully Extensible & Self-Improving:

Custom reasoning & built-in eval framework to help you rapidly iterate

02 SSMs





State Space
Models

The work around SSMs, which allow you to learn extremely long patterns and sequences without growing quadratically in computation, probably is **the next Transformer**.

$O(N^2)$

Jensen Huang
CEO
NVIDIA

State space models (SSMs) are a breakthrough for AGI

> **Transformers** are compute hungry and can't handle large context

> Like humans, **SSMs are low latency** & efficiently handle large context

> **Compute Efficient:** Transformer quality with 10-100x throughput

> **Low Latency:** Enable real-time interaction at human speeds

Long Context: Enable lifelong learning and long context models> **Long Context:** Enable lifelong learning and long context models

Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu*
Machine Learning Department
Carnegie Mellon University
agu@cs.cmu.edu

Tri Dao*
Department of Computer Science
Princeton University
tri@tridao.me

Abstract

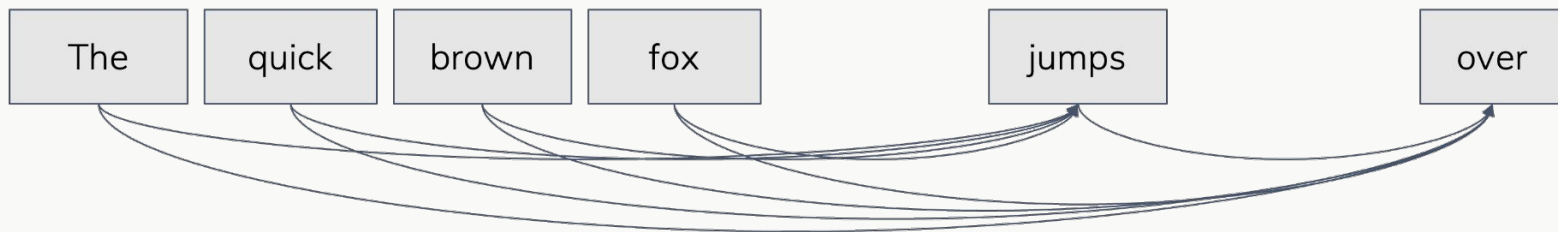
Foundation models, now powering most of the exciting applications in deep learning, are almost universally based on the Transformer architecture and its core attention module. Many subquadratic-time architectures such as linear attention, gated convolution and recurrent models, and structured state space models (SSMs) have been developed to address Transformers' computational inefficiency on long sequences, but they have not performed as well as attention on important modalities such as language. We identify that a key weakness of such models is their inability to perform content-based reasoning, and make several improvements. First, simply letting the SSM parameters be functions of the input addresses their weakness with discrete modalities, allowing the model to *selectively* propagate or forget information along the sequence length dimension depending on the current token. Second, even though this change prevents the use of efficient convolutions, we design a hardware-aware parallel algorithm in recurrent mode. We integrate these selective SSMs into a simplified end-to-end neural network architecture without attention or even MLP blocks (**Mamba**). Mamba enjoys fast inference ($5\times$ higher throughput than Transformers) and linear scaling in sequence length, and its performance improves on real data up to million-length sequences. As a general sequence model backbone, Mamba achieves state-of-the-art performance across several modalities such as language, audio, and genomics. On language modeling, our Mamba-3B model outperforms Transformers of the same size and matches Transformers twice its size, both in pretraining and downstream evaluation.

1 Introduction

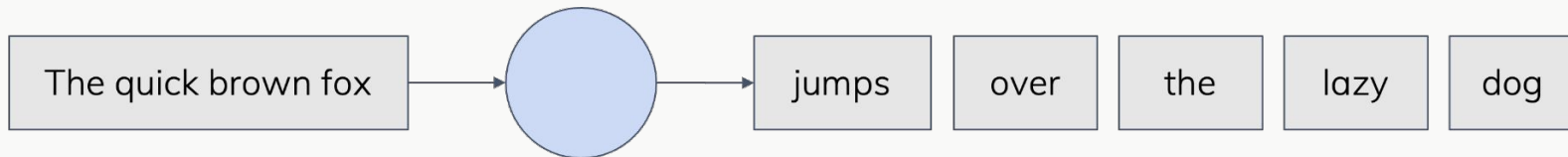
Foundation models (FMs), or large models pretrained on massive data then adapted for downstream tasks, have emerged as an effective paradigm in modern machine learning. The backbone of these FMs are often *sequence models*, operating on arbitrary sequences of inputs from a wide variety of domains such as language, images, speech, audio, time series, and genomics (Sutskever et al., 2014; Dosovitskiy et al., 2020; Oord et al., 2016; Brown et al., 2020; Ismail Fawaz et al., 2019; Poli et al., 2023). While this concept is agnostic to a particular choice of model architecture, modern FMs are predominantly based on a single type of sequence model: the Transformer (Vaswani et al., 2017) and its core attention layer. The efficacy of self-attention is attributed to its ability to route information densely within a context window, allowing it to model complex data. However, this property brings fundamental drawbacks: an inability to model anything outside of a finite window, and quadratic scaling with respect to the window length. An enormous body of research has appeared on more efficient variants of attention to overcome these drawbacks (Tay et al., 2022), but often at the expense of the very properties that makes it effective. As of yet, none of these variants have been shown to be empirically effective at scale across domains.

State Space Models (SSMs)

Transformers generate **quadratically** by attending to every past token



In SSMs, tokens stream in, update the model's state and are discarded



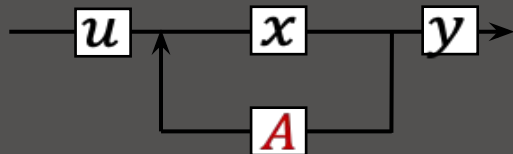
State is highly compressed, enabling generation in **linear time**

“

The work around SSMs, which allow you to learn extremely long patterns and sequences without growing quadratically in computation, probably is the next Transformer.

Jensen Huang
CEO
NVIDIA

Why state space models (SSMs) for voice AI?



$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

Latency:

Legacy architecture was not built for fast, human-level response times

Scalability:

Cheap, high-throughput inference allows us to support high volumes of concurrent users with guaranteed performance during peak usage

Advanced Reasoning:

Unlimited context lengths allow voice agents to maintain context over hours or days of interaction without voice quality degradation



Our research powers AI models across the industry

Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality

Tri Dao¹ and Albert Gu²

¹Department of Computer Science, Princeton University

²Machine Learning Department, Carnegie Mellon University
tri@tridao.me, agu@cs.cmu.edu

Abstract

While Transformers have been the main architecture behind deep learning's success in language modeling, state-space models (SSMs) such as Mamba have recently been shown to match or outperform Transformers at small to medium scale. We show that these families of models are actually quite closely related, and develop a rich framework of theoretical connections between SSMs and variants of attention, connected through various decompositions of a well-studied class of structured *separable* matrices. Our state space duality (SSD) framework allows us to design a new architecture (**Mamba-2**) whose core layer is an refinement of Mamba's selective SSM that is 2-8x faster, while continuing to be competitive with Transformers on language modeling.

1 Introduction

Transformers, in particular decoder-only models (e.g. GPT (Brown et al. 2020), Llama (Touvron, Lavril, et al. 2023)) which process input sequences in a causal fashion, are one of the main drivers of modern deep learning's success. Numerous approaches attempt to approximate the core attention layer to address its efficiency issues (Tay et al. 2022), such as scaling quadratically in sequence length during training and requiring a cache of size linear in sequence length during autoregressive generation. In parallel, a class of alternative sequence models, structured state-space models (SSMs), have emerged with linear scaling in sequence length during training and constant state size during generation. They show strong performance on long-range tasks (e.g. S4 (Gu, Goel, and Ré 2022)) and recently matched or beat Transformers on language modeling (e.g. Mamba (Gu and Dao 2023)) at small to moderate scale. However, the development of SSMs have appeared disjoint from the community's collective effort to improve Transformers, such as understanding them theoretically as well as optimizing them on modern hardware. As a result, it is more difficult to understand and experiment with SSMs compared to Transformers, and it remains challenging to train SSMs as efficiently as Transformers from both an algorithmic and systems perspective.

Our main goal is to develop a rich body of theoretical connections between structured SSMs and variants of attention. This will allow us to transfer algorithmic and systems optimizations originally developed for Transformers to SSMs, towards the goal of building foundation models that perform better than Transformers while scaling more efficiently in sequence length. A milestone contribution in this direction was the **Linear Attention (LA)** framework (Katharopoulos et al. 2020), which derived a connection between autoregressive attention and linear RNNs by showing the equivalence between "dual forms" of quadratic kernelized attention and a particular linear recurrence. This duality allows new capabilities such as the ability to have both efficient parallelizable training and efficient autoregressive inference. In the same spirit, this paper provides multiple viewpoints connecting linear-complexity SSMs with quadratic-complexity forms to combine the strengths of SSMs and attention.¹

Dynamic Chunking for End-to-End Hierarchical Sequence Modeling

Sukjun Hwang
Carnegie Mellon University
sukjunh@cs.cmu.edu

Brandon Wang
Cartesia AI
brandon.wang@cartesia.ai

Albert Gu
Carnegie Mellon University, Cartesia AI
agu@cs.cmu.edu, albert@cartesia.ai

Abstract

Major progress on language models (LMs) in recent years has largely resulted from moving away from specialized models designed for specific tasks, to general models based on powerful architectures (e.g. the Transformer) that learn everything from raw data. Despite this trend, pre-processing steps such as tokenization remain a barrier to true end-to-end foundation models. We introduce a collection of new techniques that enable a **dynamic chunking** mechanism which automatically learns content- and context- dependent segmentation strategies learned jointly with the rest of the model. Incorporating this into an explicit **hierarchical network (H-Net)** allows replacing the (implicitly hierarchical) tokenization-LM-detokenization pipeline with a single model learned fully end-to-end. When compute- and data- matched, an H-Net with one stage of hierarchy operating at the byte level outperforms a strong Transformer language model operating over BPE tokens. Iterating the hierarchy to multiple stages further increases its performance by modeling multiple levels of abstraction, demonstrating significantly better scaling with data and matching the token-based Transformer of twice its size. H-Nets pretrained on English show significantly increased character-level robustness, and qualitatively learn meaningful data-dependent chunking strategies without any heuristics or explicit supervision. Finally, the H-Net's improvement over tokenized pipelines is further increased in languages and modalities with weaker tokenization heuristics, such as Chinese and code, or DNA sequences (nearly 4x improvement in data efficiency over baselines), showing the potential of true end-to-end models that learn and scale better from unprocessed data.

1 Introduction

A broad goal of deep learning is to learn meaningful patterns from raw data, automatically extracting features and building abstractions in an end-to-end fashion. However, fixed-vocabulary **tokenization**, the process of compressing raw text into predefined chunks through algorithms such as byte-pair encoding (BPE) (Sennrich, Haddow, and Birch 2015; Kudo and Richardson 2018), remains a pervasive handcrafted preprocessing step in modern language models (LMs) (Grattafiori et al. 2024; Brown et al. 2020). Tokenization comes with a host of well-documented drawbacks, from poor character-level understanding to lack of meaning and interpretability to degraded performance on complex languages and modalities (Petrov et al. 2023; Ahia, Kumar, Gonen, Kasai, et al. 2023; Belinkov and Bisk 2017; Sun et al. 2020; Xue et al. 2022; Clark, Garrette, et al. 2022).¹ Replacing the tokenization-LM-detokenization pipeline with a single end-to-end model would also adhere better to the spirit of deep learning, ideally scaling more powerfully with data and parameters (c.f. *the bitter lesson*) (Sutton 2019; Peric 2023). However, tokenization remains an indispensable component of language models and other sequential data for its ability to compress and shorten sequences; as of yet, no *end-to-end* tokenizer-free model has matched the performance of tokenizer-based language models when matched for computational budget.



Hymba,
Nemotron



Qwen-3-Next



Minimax-M1



Jamba



Hunyuan-T1



Phi-4, Samba





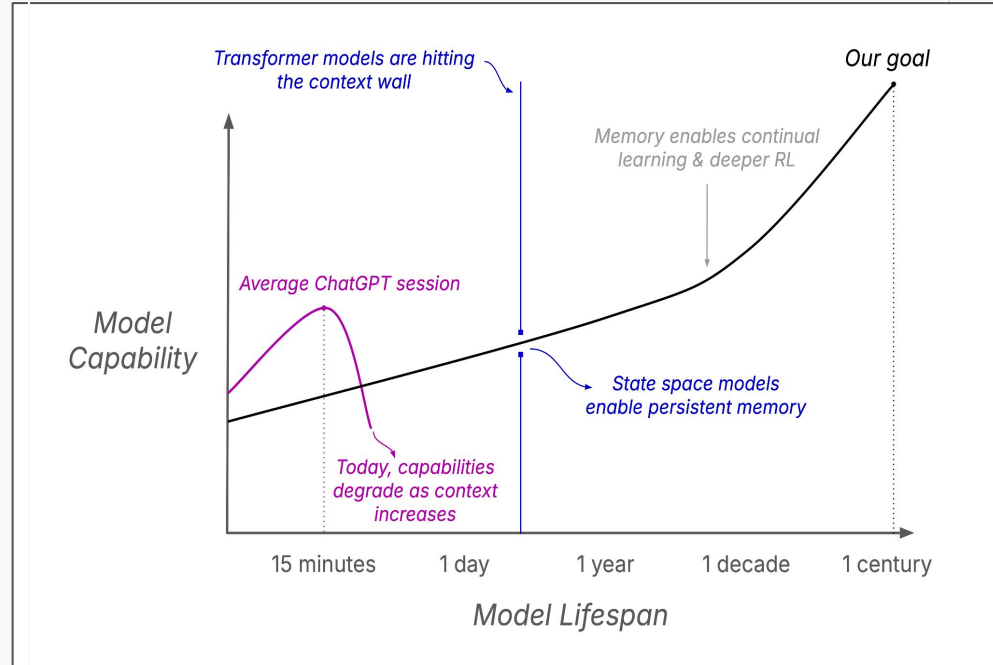
Voice Is Just the Beginning...

Transformer models hit a context wall. SSMs scale to theoretical infinite context, unlocking persistent memory that grows smarter over time.

Models that don't just respond, but **remember, learn, and adapt**.

Long memory enables:

- **Continual learning** and deeper reinforcement learning
- **Intelligent data handling** in the field
- **Autonomous systems** in high-stakes, rapidly evolving environments
- **Real-time situational awareness** and long-horizon decision-making



Building Multilingual AI Voice Agents That Customers Trust

03

AI Voice Agents

(that customers trust)

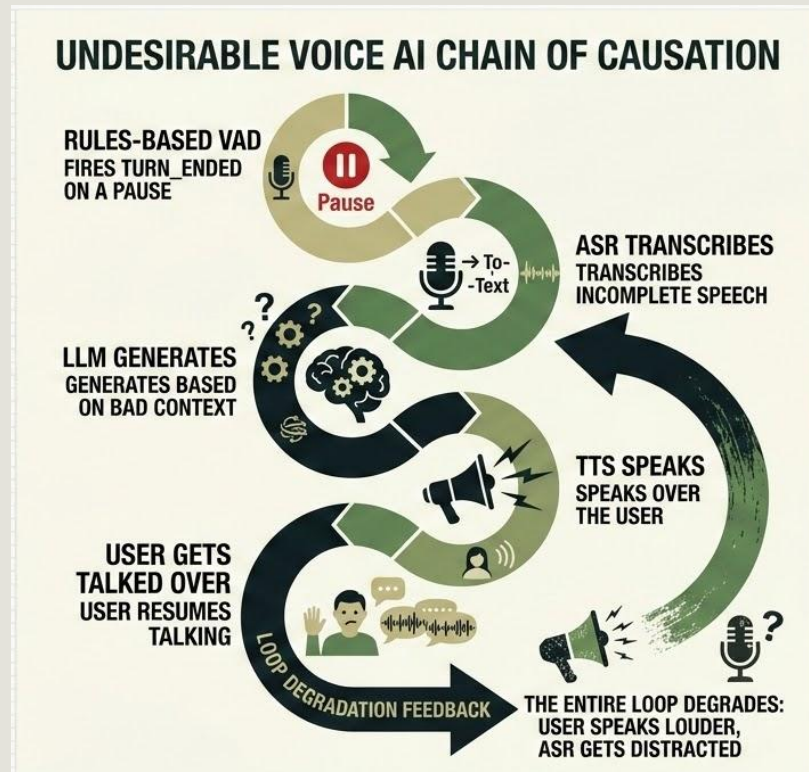
Voice Agents are not Chatbots with Voice

- > Simple STT → LLM → TTS is great on your laptop, but...
- > On 8khz PSTN (public telephony)?
- > e2e Latency (P95)
- > Conversation is messy (backchanneling, pauses, interruptions, background noise)



Voice Agents are not Chatbots with Voice Cont

- > Rules based VAD != Interruption Handling
- > Detecting speech is not enough (precision vs recall)
- > Low WER (accuracy) & low latency is not enough either
- > Clear, natural speech is not enough either!
- > Trade offs are real - use case dependent.



Full Stack Latency Budgets - Part 1

How Latency is experienced

< 500ms

Feels Conversational

Natural interaction with no conscious gap

500-800ms

Acceptable Delay

Slight pause, but flows reasonably

800ms-1.5s

Noticeably Slow

Lag begins to disrupt natural turn-taking

> 1.5s

Broken Experience

Users frequently interrupt or abandon call

Full Stack Latency Budgets - Part 2

COMPONENT	IDEAL TARGET	NOTES
VAD Turn detection	10–50ms	Endpointing accuracy is the real lever, not raw speed
STT Streaming engine	60–150ms	Batch STT adds 600-1200ms — kills the budget outright
LLM Time-to-first-token	100–250ms	Biggest, least controllable chunk; stack dependent
TTS Time-to-first-audio	40–100ms Cartesia: 40-60ms	Cartesia Sonic is fastest production TTS at 40-60ms
Network Transport layer	20-60ms WebRTC 150-700ms PSTN	PSTN telephony transport alone can blow the whole budget



Product Readiness

Quality:

For ASR: accurate, fast, and are transcripts complete?

For TTS: Does it “sound right”, “speak right”? Does the agent “do right”

Latency:

ASR: TTCT | TTS : TTFB

Latency Budgets

Turn Taking & Interruption Handling:

Handling pauses, user speech intent, turn detection, “endpointing”. Precision & Recall.





Example Use Case

Multilingual Product-Support Agent with Escalation



Building Multilingual AI Voice Agents That Customers Trust

Voice agents that customers trust have to:

- > sound right,
- > understand right,
- > respond right, and
- > recover gracefully

in every supported language.



Sonic TTS:

44 languages, 61 locales, sub-90ms TTFB



Ink-2 ASR/STT:

Super fast, highly accurate, and includes turn detection with semantic endpointing, TTFS 100ms



Line Voice Agents

voice agent infra with LLM reasoning, tool calling, and telephony



Product-Readiness

The voice agent must:

- > Listen & Understand accurately
- > Respond in the caller's locale (language, accent, normalized)
- > Handle Interruptions + Update state gracefully
- > Call tools reliably & handle failure
- > Be compliant, handle PII etc
- > Escalate intelligently
- > O11y



The Models Matter

Accuracy, speed, availability, precision vs recall



ASR

Detect speech, detect silence, understand accents, transcribe fast, handle interruptions



TTS

localization/multilingual, accents, normalization, prosody and tone

Product-Readiness (contd)

Locale Awareness:

Multilingual TTS. Language + localization. Sounds native.

X-lingual: Same voice sound like itself in other languages.

Reliability:

Normalization, alphanumerics, degraded audio, background noise, consistency across sample rates, encoding types, spliced audio, clipped syllables,



The Models Matter

Accuracy, speed, availability, precision vs recall



ASR

Detect speech, detect silence, understand accents, transcribe fast, handle interruptions



TTS

localization/multilingual, accents, normalization, prosody and tone

04 Sonic (TTS)

Languages

Indic languages

Hindi	Tamil	Telugu	Kannada	Bengali
Gujarati	Marathi	Punjabi	Malayalam	Odia
Urdu				

North American & European languages

English	Spanish	French	Czech	Croatian
German	Italian	Portuguese	Hungarian	Danish
Bulgarian	Dutch	Greek	Polish	Finnish
Georgian	Romanian	Norwegian	Russian	Slovak
Swedish	Ukrainian			

Rest of the world

Arabic	Chinese	Malay	Hebrew	Japanese
Korean	Thai	Tagalog	Indonesian	Turkish
Vietnamese				

Our languages...

- Keep the same speaker identity across languages
- Speak at real-time speed, perfect for conversational needs
- Handle complex inputs such as numbers, order IDs, and monetary amounts

Sonic in Playground





05

Ink-2 (ASR/STT)

Ink-2 in Playground



04 Line Agents



Voice Agents

- > Voice Agent Loops need orchestration
- > State machines vs linear pipelines
- > LLM in the middle
- > Lifecycle hooks
- > Tool calling - types of tools
- > Tracing & o11y
- > Managed Telephony
- > Compliance

06

On Vapi

07 Getting It Right



From Prototype to Production: A Checklist

- > Define Worst-Case Call Scenario
- > Map Use Cases to Core Metrics
- > Set Component Latency Budgets (ASR, LLM, TTS)
- > Test with Real Phone Call Audio
- > Select Deployment: Managed vs Self-Hosted
- > Localize & Verify Voice Identity
- > Define Human Escalation Paths
- > Validate Structured Data & Interruptions
- > Establish Production Monitoring Framework



Try for free + API Keys:

play.cartesia.ai

Docs:

docs.cartesia.ai

Client Libraries:

<https://github.com/cartesia-ai/cartesia-js>

<https://github.com/cartesia-ai/cartesia-python>

Credits Coupon - 1 Month of Pro
CARTESIA_VAPI2026_JL5PP

Docs Skill

docs.cartesia.ai/skill.md

Blog

<https://cartesia.ai/blog>

CARTESIA[✦]



MISTAKE VS. REALITY: BUILDING A VIABLE VOICE AGENT

THE MISTAKE: A NAIVE PIPELINE

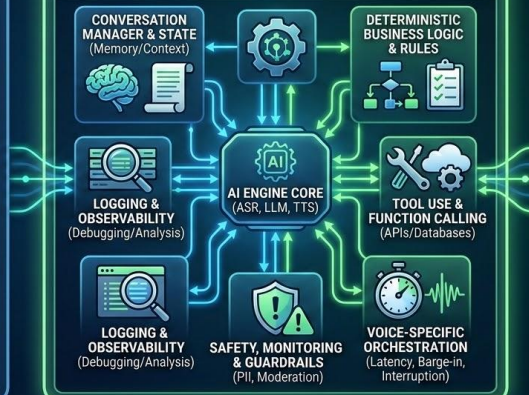


ASR → LLM → TTS = AGENT

Thinking this is a complete, viable,
and effective voice agent.

NAIVE PIPELINE

THE REALITY: COMPLEX AGENT ORCHESTRATION



THE BIG MISTAKE: WHY SIMPLE PIPELINES FAIL

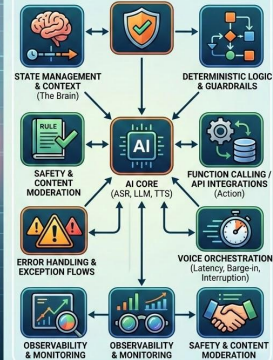
MISTAKE:
ASR → LLM → TTS = AGENT



Thinking this is a complete,
scalable, and effective voice agent.

NAIVE PIPELINE
(Chatbot with a Mic)

**REALITY: VOICE AGENT
SYSTEM ARCHITECTURE**



**REALITY: VOICE AGENT
SYSTEM ARCHITECTURE**

UNDESIRABLE VOICE AI CHAIN OF CAUSATION

